RELATIONAL DATABASE ENGINEERING • OUTAGE POST-MORTEM

When IDs Run Out: GitHub's MySQL Auto-Increment INT Overflow

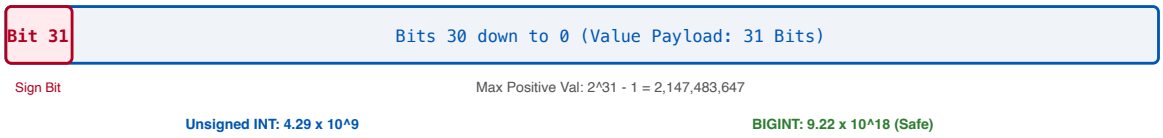
Dissection of 32-bit Integer Saturation, Primary Key Collisions, and Emergency Schema Alterations

EXECUTIVE SUMMARY

On May 5, 2020, GitHub suffered a 2.4-hour partial outage across Actions, Dependabot, and Pages when a core MySQL auto-increment primary key hit the signed 32-bit integer limit ($2^{31}-1 = 2,147,483,647$). When the ceiling was reached, the auto-increment generator clamped, causing subsequent inserts to generate primary key duplicate collisions (`ERROR 1062`) and triggering application HTTP 500 errors.

1. THE INTEGER OVERFLOW ARCHITECTURE BARRIER

In relational databases, primary keys default to 4-byte signed integers unless explicitly typed. The maximum value is strictly bounded by binary representation:



2. MECHANISM OF COLLISION: WHAT HAPPENS AT THE CEILING

When an auto-increment column hits $2,147,483,647$, MySQL does not wrap around to zero; it clamps at the maximum value:

```
// Consecutive Inserts at Boundary:
INSERT INTO events (payload) VALUES ('event_A'); -- Allocated ID: 2,147,483,647 [SUCCESS]
INSERT INTO events (payload) VALUES ('event_B'); -- Attempted ID: 2,147,483,647 [ERROR 1062]
ERROR 1062 (23000): Duplicate entry '2147483647' for key 'PRIMARY'
```

MySQL Engine Behavior

InnoDB sequence generator clamps at `INT_MAX`. Every subsequent write crashes with a primary key collision constraint violation, causing 100% write failure on the affected table.

Application Layer Impact

ActiveRecord / ORMs raise `ActiveModel::RangeError`. Background queues and HTTP endpoints abort transactions, resulting in cascading HTTP 500 error spikes across user dashboards.

3. COMPARISON: STORAGE TYPES & CAPACITY LIFESPANS

Selecting the proper column data type determines how many years a high-throughput table can operate before integer exhaustion:

SQL Data Type	Storage Width	Maximum ID Value	Exhaustion Horizon (at 10,000 writes/sec)
Signed INT	4 Bytes	$2,147,483,647$	Critical 2.48 Days (if continuous burst)
Unsigned INT	4 Bytes	$4,294,967,295$	Moderate 4.97 Days (2x Headroom)
Signed BIGINT	8 Bytes	$9,223,372,036,854,775,807$	Permanent 29,247,120 Years
Unsigned BIGINT	8 Bytes	$18,446,744,073,709,551,615$	Permanent 58,494,241 Years

4. OUTAGE INCIDENT CHRONOLOGY

Time	Component	Observed Telemetry & Anomaly	Severity
T + 00:00	Database Engine	Table sequence hits $2,147,483,647$. Next insert fails with duplicate key.	Collision First failure
T + 00:05	API & Web Workers	ActiveRecord raises `RangeError`; HTTP 500 error rates spike.	500 Errors Core Degraded
T + 00:30	DBA Team	Root cause identified as 32-bit INT exhaustion on primary table.	Triage Plan formed
T + 01:10	Emergency Alter	Executing optimized column expansion to BIGINT with disabled checks.	Executing Live Migration
T + 02:24	GitHub Systems	Migration completes; writes resume; pending queue drains to zero.	Resolved Incident Closed

5. DEEP ROOT CAUSE ANALYSIS

1. Missing Capacity Telemetry

The database monitoring stack monitored CPU, disk, and replication lag, but lacked an automated alert checking auto-increment exhaustion percentage against `information_schema`.

2. Legacy Schema Defaults

The table was provisioned years prior when 2 billion rows seemed unimaginable. Rapid platform growth accelerated write velocity beyond original architectural forecasts.

6. PRODUCTION REMEDIATION & ZERO-DOWNTIME TABLE SWAP

To permanently mitigate ID exhaustion without taking hours of table-lock downtime, reliability teams deploy asynchronous shadow migrations via `gh-ost`:

1. Shadow Table Creation

Deploy a shadow table (`_tbl_gho`) with identical indexes but primary key defined as `BIGINT NOT NULL AUTO_INCREMENT`.

2. Asynchronous Backfill

Copy historical rows in non-locking batches while replaying live binary log mutations to maintain continuous real-time synchronization.

3. Atomic Table Swap

Execute an atomic multi-table `RENAME TABLE` statement in sub-milliseconds to swap the ghost table into active production.

7. PROACTIVE MONITORING: AUTO-INCREMENT UTILIZATION SCANNER

```
-- Proactive Query to Detect Tables Approaching Limit (> 70% used):
SELECT table_name, column_name, auto_increment, max_val,
       ROUND((auto_increment / max_val) * 100, 2) AS pct_used
FROM (
  SELECT t.table_name, c.column_name, t.auto_increment,
    CASE WHEN c.column_type LIKE '%unsigned%' THEN 4294967295 ELSE 2147483647 END AS max_val
  FROM information_schema.tables t
  JOIN information_schema.columns c ON t.table_name = c.table_name
  WHERE c.extra = 'auto_increment' AND c.data_type = 'int'
) stats
WHERE (auto_increment / max_val) > 0.70;
```

8. PRODUCTION ENGINEERING CHECKLIST

Database Reliability Invariants

1. **Always default to BIGINT for surrogate keys:** The additional 4 bytes of storage per row are negligible compared to the disaster of an ID overflow outage.
2. **Never omit UNSIGNED on 32-bit INT:** If INT must be used for storage density, always specify `UNSIGNED` to double capacity.
3. **Alert at 70% threshold:** Implement automated metrics alerts when any table sequence crosses 70% of maximum column width.
4. **Verify client ORM type mapping:** Ensure client programming runtimes (Java, Go, Ruby) parse BIGINTs as 64-bit integer values to avoid client-side deserialization errors.
5. **Automate gh-ost execution:** Maintain pre-scripted gh-ost migration templates to upgrade column types under live traffic.